UNIVERSITI TEKNOLOGI MALAYSIA

Faculty of Computing

SECP3623

Database Programming

2025/2026 SEMESTER 1

Assignment 1 (10%)

Name	:	CHERYL CHEONG KAH VOON
Matric Id	:	A23CS0060
Section	:	01

Name	:	CHAU YING JIA
Matric Id	:	A23CS0213
Section	:	01

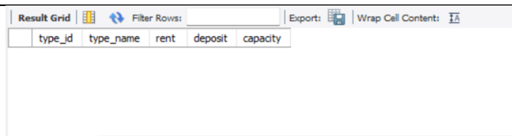
MARKING RUBRICS

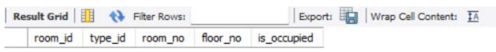
Task	Criteria	Marks	Total Marks
Task 1 Database Creation & Constraints	Creation of database and all tables with valid keys, datatypes, and constraints (`PK, FK, NOT NULL, UNIQUE`). Logical order and referential integrity preserved.	3	
	`ALTER TABLE` (add email `UNIQUE`) and `DROP TABLE` demonstration executed successfully and documented.	2	
	Insertion order, valid FK references, and no orphan data.	2	
Task 2 Data Manipulation & Filtering	Five tables populated with ≥10 valid records each; varied and meaningful data `UPDATE/DELETE` executed correctly with logical results.	1	
	Accurate use of `WHERE, BETWEEN, LIKE, IN, AND/OR/NOT` queries.	3	
	Correct use of aggregate and string functions (`COUNT, CONCAT, and more).`	2	
Task 3 Reporting & Aggregation	`VIEW` creation	2	
	Use of `GROUP BY, HAVING, ROUND, and CASE` with accurate aggregated results.	3	
	Clear labelling, readable outputs, screenshots show executed results.	2	
		Total (20)	

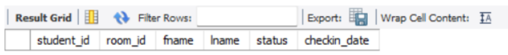
Table of Content

Task 1-Database Creation and Integrity Constraints (CLO1)

	Code	Output
1.	`CREATE DATABASE hostel_mgmt_Cheryl_YingJia;` Create a database named hostel_mgmt_Cheryl_YingJia	

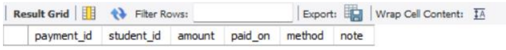
2.	`CREATE TABLE room_types (`
	`type_id int AUTO_INCREMENT PRIMARY KEY,`
	`type_name VARCHAR (255) NOT NULL,`
	`rent DECIMAL (10,2) NOT NULL,`
	`deposit DECIMAL (10,2) NOT NULL,`
	`capacity INT NOT NULL`
	a. `);`
	Create table named room_types with the entity type_id, type_name, rent, deposit and capacity
		

	`CREATE TABLE rooms (`
	`room_id int AUTO_INCREMENT PRIMARY KEY,`
	`type_id int ,`
	`room_no VARCHAR (50) NOT NULL,`
	`floor_no int NOT NULL,`
	`is_occupied BOOLEAN NOT NULL,`
	`FOREIGN KEY (type_id) REFERENCES room_types(type_id)`
	b. `);`
	Create table named rooms with the entity room_id, type_id, room_no, floor_no and is_occupied.
	The attribute `type_id` is used as a foreign key to reference the `room_types` table.
		b.
		

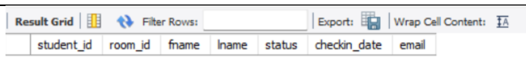
	`CREATE TABLE students (`
	`student_id int AUTO_INCREMENT PRIMARY KEY,`
	`room_id int NOT NULL,`
	`fname VARCHAR (255) NOT NULL,`
	`lname VARCHAR (255) NOT NULL,`
	`status ENUM('ACTIVE' , 'NON_ACTIVE') NOT NULL,`
	`checkin_date DATE,`
	`FOREIGN KEY (room_id) REFERENCES rooms (room_id)`
	c. `);`
	Create table named students with the entity student_id, room_id, fname, lname, status and checkin_date.
	The attribute `room_id` is used as a foreign key to reference the rooms table.
		c.
		

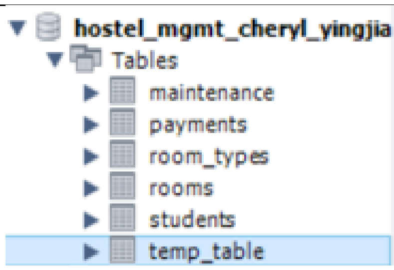
	`CREATE TABLE maintenance (`
	`maint_id int AUTO_INCREMENT PRIMARY KEY,`
	`room_id int NOT NULL,`
	`issue_desc TEXT NOT NULL,`
	`severity ENUM ('LOW', 'MEDIUM', 'HIGH') NOT NULL,`
	`status ENUM ('OPEN','RESOLVED') NOT NULL,`
	`reported_on DATE NOT NULL,`
	`resolved_on DATE NULL,`
	`FOREIGN KEY (room_id) REFERENCES rooms (room_id)`
	`);`
	d.
		d.
		

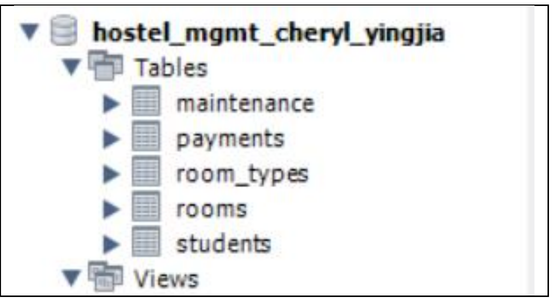
	Create table named maintenance with the entity maint_id, room_id, issue_desc, severity, status, reported_on and resolved_on. The attribute `room_id` is used as a foreign key to reference the `rooms` table.	

	`CREATE TABLE payments (`
	`payment_id int AUTO_INCREMENT PRIMARY KEY,`
	`student_id int NOT NULL,`
	`amount DECIMAL (10,2),`
	`paid_on DATE NOT NULL,`
	`method ENUM ('CASH','FPX','CARD','TNG') NOT NULL,`
	`note VARCHAR (255),`
	`FOREIGN KEY (student_id) REFERENCES students (student_id)`
	`);`
	e.
	Create table named payments with the entity payment_id, student_id, amount, paid_on, method and note.
	The attribute `student_id` is used as a foreign key to reference the `students` table.
		e.
		

4.	• `UPDATE rooms r`
	`SET is_occupied = EXISTS (`
	`SELECT 1`
	`FROM students s`
	`WHERE s.room_id = r.room_id`
	`AND s.status = 'ACTIVE'`
	`)`
	`WHERE r.room_id > 0;`
	A room is ‘occupied’ if it has ≥1 student with status='ACTIVE'; otherwise 'VACANT'	

5.	• `ALTER TABLE students` `ADD email VARCHAR (255) UNIQUE;` Use ALTER TABLE to add a new column email to the `students` table and mark it as UNIQUE	

	`CREATE TABLE temp_table (`
	`temp_id int`
	`);`
	
	`DROP TABLE IF EXISTS temp_table;`
	Create a temporary table named temp_table and remove it safely using DROP TABLE IF EXISTS
		

		hostel_mgmt_cheryl_yingjia Tables maintenance payments room_types rooms students Views

Task 2 - Data Manipulation and Filtering (CLO2)

	Code	Output

1.

```sql
INSERT INTO room_types(type_name, rent, deposit, capacity)
VALUES ('Single Standard', 300.00, 150.00, 1),
       ('Double Standard', 400.00, 200.00, 2),
       ('Single Deluxe', 400.00, 200.00, 1),
       ('Double Deluxe', 500.00, 250.00, 2),
       ('Single Premium', 500.00, 250.00, 1),
       ('Double Premium', 600.00, 300.00, 2),
       ('Family Standard (Small)', 900.00, 500.00, 4),
       ('Family Standard (Large)', 1200.00, 600.00, 6),
       ('Family Deluxe (Small)', 1100.00, 300.00, 1),
       ('Family Deluxe (Large)', 1400.00, 250.00, 2);
```

Insert 10 data into room_types table. Different room types for single, double and family rooms. We have set the type_id to auto increment.

type_id	type_name	rent	deposit	capacity
1	Single Standard	300.00	150.00	1
2	Double Standard	400.00	200.00	2
3	Single Deluxe	400.00	200.00	1
4	Double Deluxe	500.00	250.00	2
5	Single Premium	500.00	250.00	1
6	Double Premium	600.00	300.00	2
7	Family Standard (Small)	900.00	500.00	4
8	Family Standard (Large)	1200.00	600.00	6
9	Family Deluxe (Small)	1100.00	300.00	1
10	Family Deluxe (Large)	1400.00	250.00	2

```sql
INSERT INTO rooms (type_id, room_no, floor_no, is_occupied)
VALUES (1, 'A101', 1, FALSE),
       (2, 'B111', 1, FALSE),
       (3, 'A201', 2, FALSE),
       (4, 'B211', 2, FALSE),
       (5, 'A301', 3, FALSE),
       (6, 'B311', 3, FALSE),
       (7, 'C401', 4, FALSE),
       (8, 'C411', 4, FALSE),
       (9, 'C501', 5, FALSE),
       (10, 'C512', 5, FALSE);
```

Insert 10 data into rooms table. We follow the sequence for type_id then we assign room number, floor number and occupant of rooms.

room_id	type_id	room_no	floor_no	is_occupied
1	1	A101	1	0
2	2	B111	1	0
3	3	A201	2	0
4	4	B211	2	0
5	5	A301	3	0
6	6	B311	3	0
7	7	C401	4	0
8	8	C411	4	0
9	9	C501	5	0
10	10	C512	5	0

```sql
INSERT INTO students (room_id, fname, lname, status, checkin_date, email)
VALUES (1, 'Ahmad', 'Mohammed', 'ACTIVE', '2025-08-02', 'ahmadm@graduate.utm.my'),
       (2, 'Yee Wen', 'Lau', 'ACTIVE', '2025-09-01', 'yeewen@graduate.utm.my'),
       (3, 'Cheryl', 'Cheong', 'ACTIVE', '2025-08-15', 'cheryl@graduate.utm.my'),
       (4, 'Dora', 'Poh', 'ACTIVE', '2025-09-05', 'pohdora@graduate.utm.my'),
       (5, 'Emily', 'Liew', 'ACTIVE', '2025-08-31', 'emily_l@graduate.utm.my'),
       (6, 'Jeffery', 'Gan', 'NON_ACTIVE', '2024-09-22', 'jeff@graduate.utm.my'),
       (7, 'Ganesh', 'Kumar', 'ACTIVE', '2025-10-27', 'ganesh_k@graduate.utm.my'),
       (8, 'Wyness', 'Ong', 'ACTIVE', '2025-08-17', 'wyness@graduate.utm.my'),
       (9, 'Ibrahim', 'Ali', 'NON_ACTIVE', '2024-10-23', 'ibrahimali@graduate.utm.my'),
       (10, 'Jasmine', 'Lee', 'ACTIVE', '2025-09-10', 'jasmine@graduate.utm.my');
```

Insert 10 data into students table. We assigned students into rooms accordingly also with their name, status, check-in dates and their email.

student_id	room_id	fname	lname	status	checkin_date
1	1	Ahmad	Mohammed	ACTIVE	2025-08-02
2	2	Yee Wen	Lau	ACTIVE	2025-09-01
3	3	Cheryl	Cheong	ACTIVE	2025-08-15
4	4	Dora	Poh	ACTIVE	2025-09-05
5	5	Emily	Liew	ACTIVE	2025-08-31
6	6	Jeffery	Gan	NON_ACTIVE	2024-09-22
7	7	Ganesh	Kumar	ACTIVE	2025-10-27
8	8	Wyness	Ong	ACTIVE	2025-08-17
9	9	Ibrahim	Ali	NON_ACTIVE	2024-10-23
10	10	Jasmine	Lee	ACTIVE	2025-09-10

```sql
INSERT INTO maintenance (room_id, issue_desc, severity, status, reported_on, resolved_on)
VALUES(1, 'Water leakage in bathroom', 'MEDIUM', 'OPEN', '2025-10-10', NULL),
      (2, 'Broken cabinet', 'LOW', 'RESOLVED', '2025-09-15', '2025-09-16'),
      (5, 'Aircond not cold', 'HIGH', 'OPEN', '2025-11-01', NULL),
      (10, 'Window latch broken', 'LOW', 'OPEN', '2025-11-03', NULL),
      (7, 'Clogged toilet', 'HIGH', 'RESOLVED', '2025-10-05', '2025-10-05'),
      (4, 'Faulty wiring for study lamp', 'MEDIUM', 'OPEN', '2025-10-28', NULL),
      (2, 'Bed frame cracked', 'MEDIUM', 'RESOLVED', '2025-09-20', '2025-09-22'),
      (8, 'Wifi router not working', 'LOW', 'OPEN', '2025-11-05', NULL),
      (1, 'Shower heater broken', 'MEDIUM', 'RESOLVED', '2025-05-01', '2025-05-04'),
      (3, 'Door lock jammed', 'MEDIUM', 'RESOLVED', '2025-06-15', '2025-06-16'),
      (3, 'lamp not function', 'LOW', 'OPEN', '2025-06-15', NULL),
      (5, 'Window cannot open', 'LOW', 'OPEN', '2025-06-15', NULL),
      (4, 'Bed frame cracked', 'MEDIUM', 'OPEN', '2025-06-15', NULL),
      (6, 'Wifi router not working', 'LOW', 'OPEN', '2025-06-15', NULL);
```

Insert 14 data into maintenance table. Based on the room_id, we can know the issue occurs, the severity, the status, the reported

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	1	Water leakage in bathroom	MEDIUM	OPEN	2025-10-10	NULL
2	2	Broken cabinet	LOW	RESOLVED	2025-09-15	2025-09-16
3	5	Aircond not cold	HIGH	OPEN	2025-11-01	NULL
4	10	Window latch broken	LOW	OPEN	2025-11-03	NULL
5	7	Clogged toilet	HIGH	RESOLVED	2025-10-05	2025-10-05
6	4	Faulty wiring for study lamp	MEDIUM	OPEN	2025-10-28	NULL
7	2	Bed frame cracked	MEDIUM	RESOLVED	2025-09-20	2025-09-22
8	8	Wifi router not working	LOW	OPEN	2025-11-05	NULL
9	1	Shower heater broken	MEDIUM	RESOLVED	2025-05-01	2025-05-04
10	3	Door lock jammed	MEDIUM	RESOLVED	2025-06-15	2025-06-16
11	3	lamp not function	LOW	OPEN	2025-06-15	NULL
12	5	Window cannot open	LOW	OPEN	2025-06-15	NULL
13	4	Bed frame cracked	MEDIUM	OPEN	2025-06-15	NULL
14	6	Wifi router not working	LOW	OPEN	2025-06-15	NULL

dates and the resolved date of the maintenance. The date of maintenance could be null when maintenance have not complete.

```sql
INSERT INTO payments (student_id, amount, paid_on, method, note)
VALUES
-- August Payments
(1, 300.00, '2025-08-03', 'FPX', 'August Rent'),  -- Ahmad (Checked in 8/2)
(3, 400.00, '2025-08-15', 'CARD', 'August Rent'), -- Cheryl (Checked in 8/15)
(8, 1200.00, '2025-08-18', 'TNG', 'August Rent'), -- Wyness (Checked in 8/17)ZS
(5, 500.00, '2025-08-31', 'FPX', 'August Rent'),  -- Emily (Checked in 8/31)

-- September Payments (for students checked in by Sep)
(1, 300.00, '2025-09-01', 'FPX', 'September Rent'), -- Ahmad (Checked in agian)
(2, 400.00, '2025-09-02', 'CARD', 'September Rent'), -- Yee Wen (Checked in 9/1)
(10, 1400.00, '2025-09-10', 'TNG', 'September Rent'), -- Jasmine (Checked in 9/10)
(4, 500.00, '2025-09-08', 'FPX', 'September Rent'),

-- October Payments (for students checked in by Oct)
(1, 300.00, '2025-10-01', 'FPX', 'October Rent'), -- Ahmad
(7, 900.00, '2025-10-27', 'CASH', 'October Rent'); -- Ganesh (Checked in 10/27)
```

Insert 10 data into payments table. Based on the above tables, calculate the amount for rent of students with the payment date, method and notes.

payment_id	student_id	amount	paid_on	method	note
1	1	300.00	2025-08-03	FPX	August Rent
2	3	400.00	2025-08-15	CARD	August Rent
3	8	1200.00	2025-08-18	TNG	August Rent
4	5	500.00	2025-08-31	FPX	August Rent
5	1	300.00	2025-09-01	FPX	September Rent
6	2	400.00	2025-09-02	CARD	September Rent
7	10	1400.00	2025-09-10	TNG	September Rent
8	4	500.00	2025-09-08	FPX	September Rent
9	1	300.00	2025-10-01	FPX	October Rent
10	7	900.00	2025-10-27	CASH	October Rent

2.

```sql
UPDATE rooms r
SET is_occupied = EXISTS (
    SELECT 1
    FROM students s
    WHERE s.room_id = r.room_id
      AND s.status = 'ACTIVE'
)
WHERE r.room_id > 0;
```

Update the is_occupied column in rooms to TRUE for rooms with ACTIVE students; otherwise, set it to FALSE. We use 1 for TRUE and 0 for FALSE.

room_id	type_id	room_no	floor_no	is_occupied
1	1	A101	1	1
2	2	B111	1	1
3	3	A201	2	1
4	4	B211	2	0
5	5	A301	3	1
6	6	B311	3	0
7	7	C401	4	1
8	8	C411	4	1
9	9	C501	5	0
10	10	C512	5	1

```sql
DELETE FROM maintenance
WHERE
    status = 'RESOLVED'
    AND reported_on < DATE_SUB(CURDATE(), INTERVAL 60 DAY);
```

Delete maintenance records where status = 'RESOLVED' and the reported date is 60 days before the current date using date subtraction.

Before:

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	1	Water leakage in bathroom	MEDIUM	OPEN	2025-10-10	NULL
2	2	Broken cabinet	LOW	RESOLVED	2025-09-15	2025-09-16
3	5	Aircond not cold	HIGH	OPEN	2025-11-01	NULL
4	10	Window latch broken	LOW	OPEN	2025-11-03	NULL
5	7	Clogged toilet	HIGH	RESOLVED	2025-10-05	2025-10-05
6	4	Faulty wiring for study lamp	MEDIUM	OPEN	2025-10-28	NULL
7	2	Bed frame cracked	MEDIUM	RESOLVED	2025-09-20	2025-09-22
8	8	Wifi router not working	LOW	OPEN	2025-11-05	NULL
9	1	Shower heater broken	MEDIUM	RESOLVED	2025-05-01	2025-05-04
10	3	Door lock jammed	MEDIUM	RESOLVED	2025-06-15	2025-06-16
11	3	lamp not function	LOW	OPEN	2025-06-15	NULL
12	5	Window cannot open	LOW	OPEN	2025-06-15	NULL
13	4	Bed frame cracked	MEDIUM	OPEN	2025-06-15	NULL
14	6	Wifi router not working	LOW	OPEN	2025-06-15	NULL

After:

maint_id	room_id	issue_desc	severity	status	reported_on	resolved_on
1	1	Water leakage in bathroom	MEDIUM	OPEN	2025-10-10	NULL
2	2	Broken cabinet	LOW	RESOLVED	2025-09-15	2025-09-16
3	5	Aircond not cold	HIGH	OPEN	2025-11-01	NULL
4	10	Window latch broken	LOW	OPEN	2025-11-03	NULL
5	7	Clogged toilet	HIGH	RESOLVED	2025-10-05	2025-10-05
6	4	Faulty wiring for study lamp	MEDIUM	OPEN	2025-10-28	NULL
7	2	Bed frame cracked	MEDIUM	RESOLVED	2025-09-20	2025-09-22
8	8	Wifi router not working	LOW	OPEN	2025-11-05	NULL
9	1	Shower heater broken	MEDIUM	RESOLVED	2025-05-01	2025-05-04
10	3	Door lock jammed	MEDIUM	RESOLVED	2025-06-15	2025-06-16
11	3	lamp not function	LOW	OPEN	2025-06-15	NULL
12	5	Window cannot open	LOW	OPEN	2025-06-15	NULL

3.

```sql
SELECT * FROM room_types
WHERE rent BETWEEN 400 AND 800;
```

Display only the rows with rent between 400 and 800 from room_types table.

type_id	type_name	rent	deposit	capacity
2	Double Standard	400.00	200.00	2
3	Single Deluxe	400.00	200.00	1
4	Double Deluxe	500.00	250.00	2
5	Single Premium	500.00	250.00	1
6	Double Premium	600.00	300.00	2
NULL	NULL	NULL	NULL	NULL

```sql
SELECT * FROM students
WHERE fname LIKE 'A%';
```

Display only the rows that first name (fname) starts with A from students table.

student_id	room_id	fname	lname	status	checkin_date	email
1	1	Ahmad	Mohammed	ACTIVE	2025-08-02	ahmadm@graduate.utm.my
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```sql
SELECT * FROM payments
WHERE method IN ('FPX','CARD');
```

Display only the rows where the method column is with 'FPX' or 'CARD' from payments table.

payment_id	student_id	amount	paid_on	method	note
11	1	300.00	2025-08-03	FPX	August Rent
12	3	400.00	2025-08-15	CARD	August Rent
14	5	500.00	2025-08-31	FPX	August Rent
15	1	300.00	2025-09-01	FPX	September Rent
16	2	400.00	2025-09-02	CARD	September Rent
18	4	500.00	2025-09-08	FPX	September Rent
19	1	300.00	2025-10-01	FPX	October Rent
NULL	NULL	NULL	NULL	NULL	NULL

```sql
SELECT * FROM payments
WHERE (amount BETWEEN 300 AND 800 AND NOT method = 'TNG')
   OR method = 'CARD';
```

Display only the amount where it is between 300 and 800 but not the method with 'TNG'. However, it can be method with 'CARD'.

payment_id	student_id	amount	paid_on	method	note
11	1	300.00	2025-08-03	FPX	August Rent
12	3	400.00	2025-08-15	CARD	August Rent
14	5	500.00	2025-08-31	FPX	August Rent
15	1	300.00	2025-09-01	FPX	September Rent
16	2	400.00	2025-09-02	CARD	September Rent
18	4	500.00	2025-09-08	FPX	September Rent
19	1	300.00	2025-10-01	FPX	October Rent
NULL	NULL	NULL	NULL	NULL	NULL

4.

a. Count()

```sql
SELECT COUNT(*) FROM students
WHERE status = 'ACTIVE'
```

Count the number of students where the status is stated 'ACTIVE' from students table.

b. SUM()

```sql
SELECT SUM(amount) FROM payments
WHERE method = 'FPX';
```

Calculate the total amount of the payments table where the method is 'FPX'.

COUNT(*)
8

a.

SUM(amount)
1900.00

b.

a. CONCAT()

```
SELECT CONCAT(lname, ' ',fname) AS Full_Name
FROM students;
```

We join the last name (lname) and the first name (fname) from students table into a new column names ‘Full_Name’ using CONCAT string function.

b. UPPER()

```
SELECT UPPER(lname) AS UpperLastName
FROM students
```

We use UPPER function to make the last name (lname) in students table into uppercase in a new column named ‘UpperLastName’.

Full_Name
Mohammed Ahmad
Lau Yee Wen
Cheong Cheryl
Poh Dora
Liew Emily
Gan Jeffery
Kumar Ganesh
Ong Wyness
Ali Ibrahim
Lee Jasmine

a.

UpperLastName
MOHAMMED
LAU
CHEONG
POH
LIEW
GAN
KUMAR
ONG
ALI
LEE

b.

Task 3 – Reporting and Aggregation (CLO2)

	Code	Output

1.

Code

```sql
CREATE OR REPLACE VIEW v_room_status AS
SELECT
    r.room_no,
    rt.type_name,
    rt.rent,
    r.floor_no,
    rt.capacity,

    (SELECT COUNT(*)
     FROM students s
     WHERE s.room_id = r.room_id AND s.status = 'ACTIVE') AS n_occupants,

    (SELECT COUNT(*)
     FROM maintenance m
     WHERE m.room_id = r.room_id AND m.status = 'OPEN') AS pending_issues,

    CASE
        WHEN (SELECT COUNT(*)
              FROM students s
              WHERE s.room_id = r.room_id AND s.status = 'ACTIVE') = 0
        THEN 'TRUE'
        ELSE 'FALSE'
    END AS is_vacant

FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id;
```

Create a view named v_room_status with the columns room_no, type_name, rent, floor_no, capacity, n_occupants, pending_issues, and is_vacant. The value n_occupants represents the count of students with status = ‘ACTIVE’ in each room. The value pending_issues represents the count of maintenance records with status = ‘OPEN’ per room. The value is_vacant is TRUE when n_occupants = 0, otherwise FALSE.

Output

room_no	type_name	rent	floor_no	capacity	n_occupants	pending_issues	is_vacant
A101	Single Standard	300.00	1	1	1	1	FALSE
B111	Double Standard	400.00	1	2	1	0	FALSE
A201	Single Deluxe	400.00	2	1	1	0	FALSE
B211	Double Deluxe	500.00	2	2	1	1	FALSE
A301	Single Premium	500.00	3	1	1	1	FALSE
B311	Double Premium	600.00	3	2	0	0	TRUE
C401	Family Standard (Small)	900.00	4	4	1	0	FALSE
C411	Family Standard (Large)	1200.00	4	6	1	1	FALSE
C501	Family Deluxe (Small)	1100.00	5	1	0	0	TRUE
C512	Family Deluxe (Large)	1400.00	5	2	1	1	FALSE

2.

Code

```sql
SELECT
    rt.type_name,
    COUNT(s.student_id) AS Total_Student
FROM room_types rt
JOIN rooms r ON rt.type_id = r.type_id
LEFT JOIN students s ON r.room_id = s.room_id
GROUP BY type_name;
```

Retrieve the total number of students grouped by room type.

Output

type_name	Total_Student
Single Standard	1
Double Standard	1
Single Deluxe	1
Double Deluxe	1
Single Premium	1
Double Premium	1
Family Standard (Small)	1
Family Standard (Large)	1
Family Deluxe (Small)	1
Family Deluxe (Large)	1

```sql
SELECT
    type_name,
    ROUND(AVG(rent),2) AS Avg_Rent,
    ROUND(SUM(deposit),2) AS Total_Deposit
FROM room_types
GROUP BY type_name;
```

Retrieve the average rent and total deposit per room type.
The values are rounded to 2 decimal places using ROUND()

type_name	Avg_Rent	Total_Deposit
Single Standard	300.00	150.00
Double Standard	400.00	200.00
Single Deluxe	400.00	200.00
Double Deluxe	500.00	250.00
Single Premium	500.00	250.00
Double Premium	600.00	300.00
Family Standard (Small)	900.00	500.00
Family Standard (Large)	1200.00	600.00
Family Deluxe (Small)	1100.00	300.00
Family Deluxe (Large)	1400.00	250.00

```sql
SELECT
    YEAR(paid_on) AS Year,
    MONTH(paid_on) AS Month,
    SUM(amount) AS Total_Amount
FROM payments
GROUP BY YEAR(paid_on) , MONTH(paid_on)
ORDER BY Year, Month;
```

Retrieve total payment amounts grouped by year and month.

Year	Month	Total_Amount
2025	8	2400.00
2025	9	2600.00
2025	10	1200.00

```sql
SELECT
    r.floor_no,
    COUNT(m.maint_id) AS open_issues
FROM maintenance m
JOIN rooms r
    ON m.room_id = r.room_id
WHERE m.status = 'OPEN'
GROUP BY r.floor_no
HAVING COUNT(m.maint_id) > 2;
```

etrieve the count of OPEN maintenance issues per floor.
Only shows floors with more than 2 issues using HAVING COUNT > 2.

floor_no	open_issues
3	3
2	3

3.

```sql
SELECT
    UPPER(rt.type_name) AS ROOM_TYPE,
    CONCAT('Floor ', r.floor_no, ', Room ', r.room_no) AS FULL_LOCATION,
    ROUND(rt.rent, 2) AS RENT_AMOUNT,

    CASE
        WHEN rt.rent < 500 THEN 'LOW'
        WHEN rt.rent BETWEEN 500 AND 800 THEN 'MEDIUM'
        ELSE 'HIGH'
    END AS RENT_CATEGORY

FROM rooms r
JOIN room_types rt ON r.type_id = rt.type_id
ORDER BY rt.rent, r.floor_no;
```

We make the room type to uppercase, join the floor number and room number together in a

ROOM_TYPE	FULL_LOCATION	RENT_AMOUNT	RENT_CATEGORY
SINGLE STANDARD	Floor 1, Room A101	300.00	LOW
DOUBLE STANDARD	Floor 1, Room B111	400.00	LOW
SINGLE DELUXE	Floor 2, Room A201	400.00	LOW
DOUBLE DELUXE	Floor 2, Room B211	500.00	MEDIUM
SINGLE PREMIUM	Floor 3, Room A301	500.00	MEDIUM
DOUBLE PREMIUM	Floor 3, Room B311	600.00	MEDIUM
FAMILY STANDARD (SMALL)	Floor 4, Room C401	900.00	HIGH
FAMILY DELUXE (SMALL)	Floor 5, Room C501	1100.00	HIGH
FAMILY STANDARD (LARGE)	Floor 4, Room C411	1200.00	HIGH
FAMILY DELUXE (LARGE)	Floor 5, Room C512	1400.00	HIGH

	column named ‘FULL_LOCATION’, round up the rental amount into 2 decimal places and show the price category. All in ascending order.	